

Analisi e Progettazione del Software

Diagrammi di macchina a stati

Oliviero Riganelli

Macchine a Stati

- Le macchine a stati possono essere utilizzate per modellare il comportamento dinamico di classificatori quali classi, casi d'uso, sottosistemi e interi sistemi.
- Le macchine a stati esistono nel contesto di un particolare classificatore che:
 - *Risponde a eventi esterni;*
 - *Ha un ciclo di vita definito, che può essere modellato come una successione di stati, transizioni ed eventi (come vedremo fra poco!)*
 - *Può avere un comportamento corrente che dipende dai comportamenti precedenti.*
- Le macchine a stati di solito vengono utilizzate per modellare il comportamento dinamico degli oggetti

Tipi di oggetti

- Indipendente dallo stato
 - *Risponde sempre nello stesso modo ad un determinato evento*
- Dipendente dallo stato
 - *Risponde in modi diversi ad un determinato evento a seconda dello stato in cui si trova*

Modellazione oggetti dipendenti dallo stato

- Modellano il comportamento di un oggetto reattivo complesso in risposta agli eventi

oppure

- Modellano le sequenze valide delle operazioni ovvero specifiche di protocollo o di linguaggio
 - *Può essere considerato una specializzazione del primo se l'oggetto e' un protocollo o un processo*

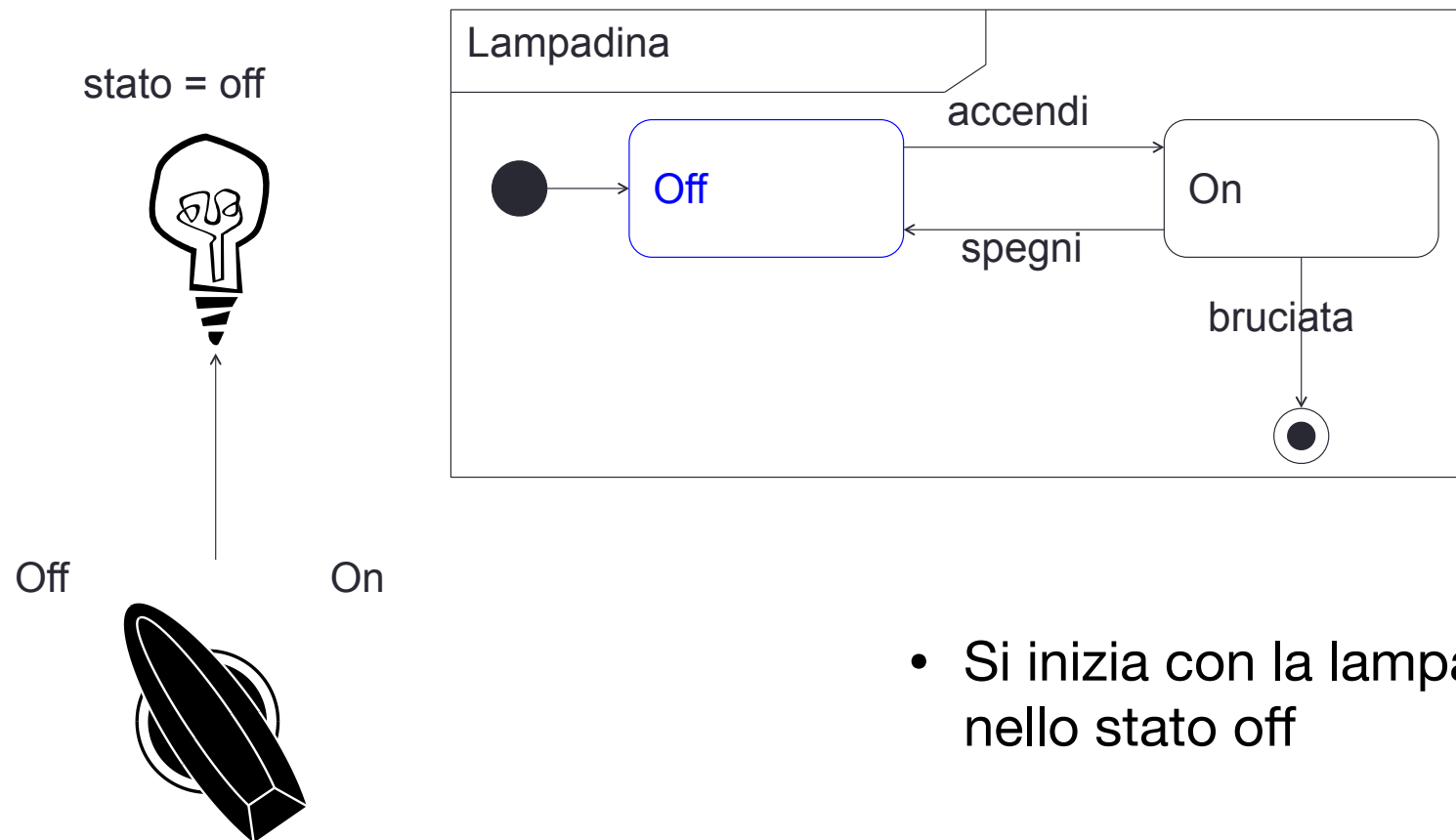
Esempi di oggetti reattivi complessi

- Dispositivi fisici (telefono, auto, forno a microonde, ecc.)
- Transazioni e oggetti di business (ordine, vendita, pagamento, ecc.)
- Mutatori di ruolo (persona da celibe a coniugato ecc.)

Esempi di protocolli e sequenze validi

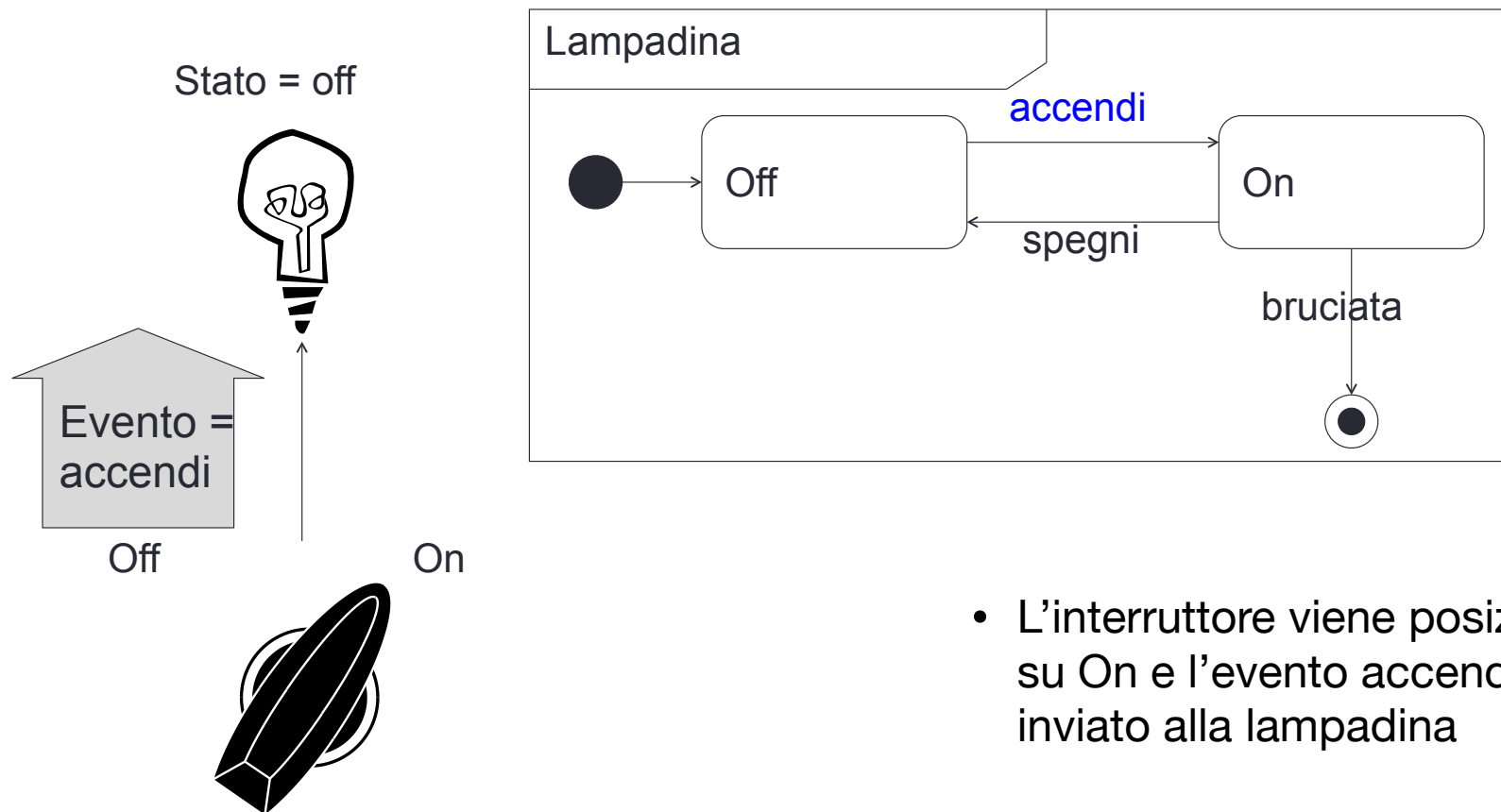
- Protocolli di comunicazione
 - *TCP: una richiesta “TCP close” deve essere ignorata se il gestore di protocollo e’ gia’ nello stato “closed”*
- Flusso e navigazione delle pagine/finestre UI
 - *Mostrano la sequenza delle pagine Web*
- Controller di flusso UI e sessioni
 - *Mostrano oggetti lato server che rappresentano una sessione o una conversazione con un client*
- Operazioni di sistema dei casi d’uso
 - *endSale deve arrivare solo dopo una o più operazione enterItem*
- Gestione degli eventi di una singola finestra UI
 - *Mostrano gli eventi e le sequenze valide per una finestra (modifica-incolla)*

Diagrammi delle macchine a stati



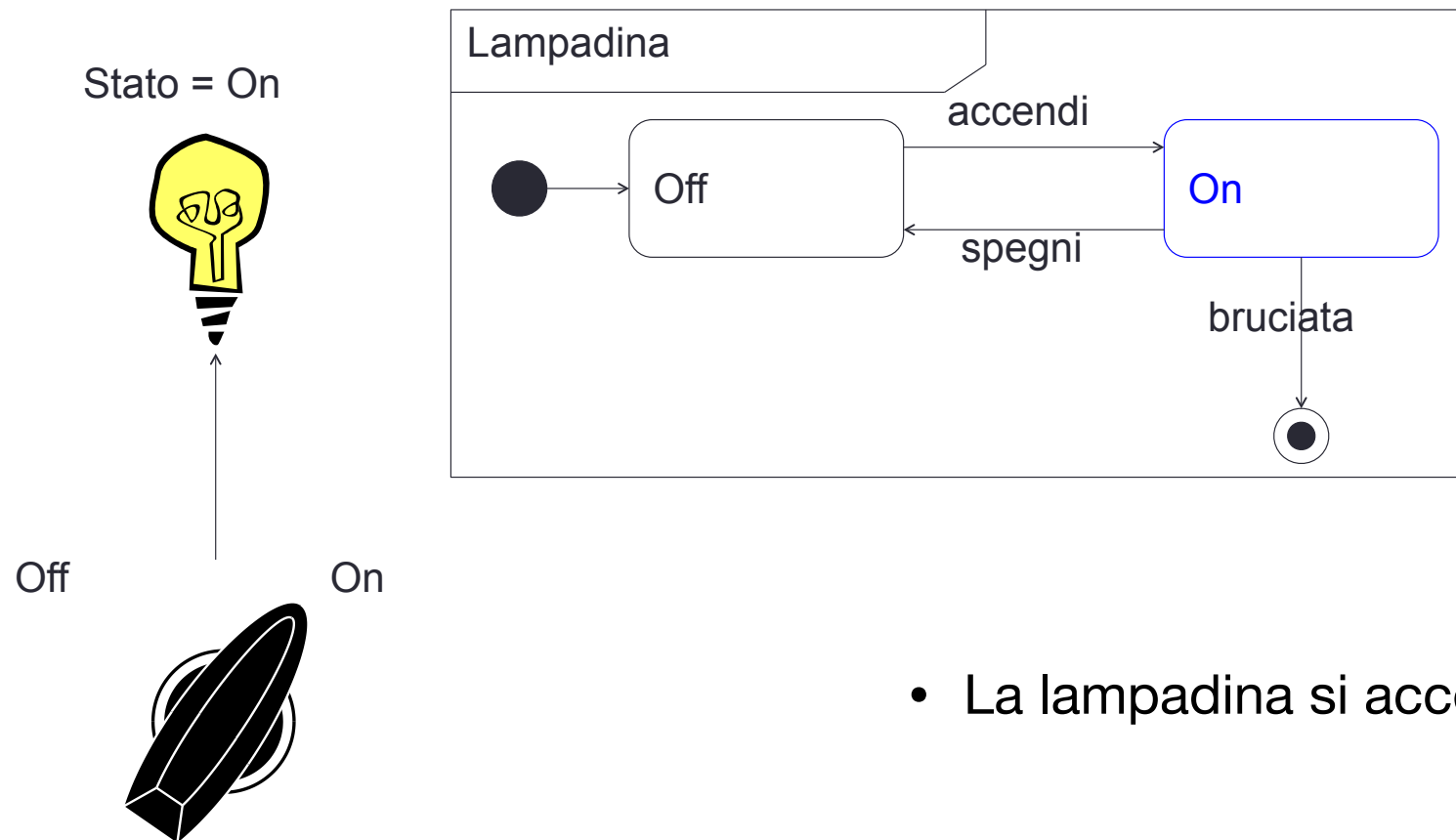
- Si inizia con la lampadina nello stato off

Accedi lampadina



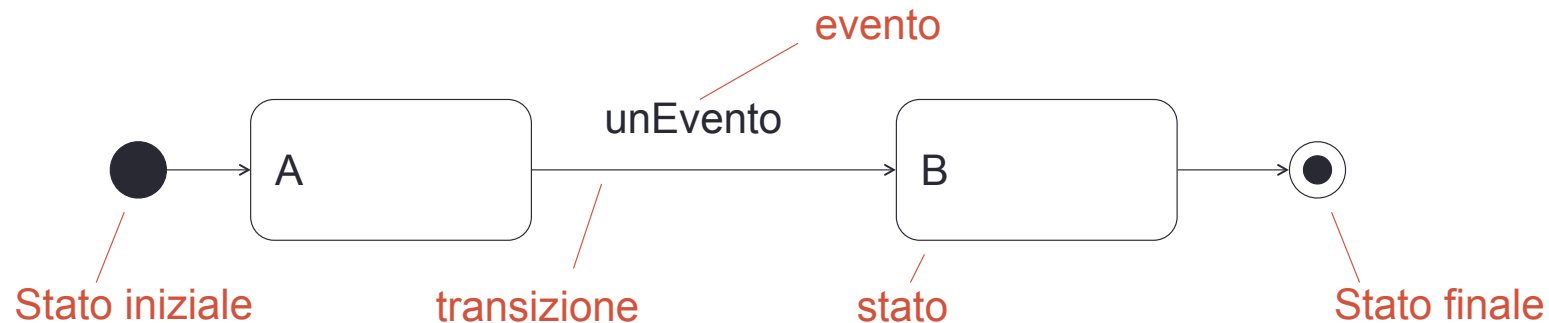
- L'interruttore viene posizionato su On e l'evento accendi è inviato alla lampadina

Lampadina Accesa



- La lampadina si accende

Sintassi base delle macchine a stati



- Ogni macchina a stati dovrebbe avere uno stato iniziale che indica il primo stato della sequenza
- A meno che esista un ciclo perpetuo di stati, dovrebbe avere uno stato finale che termina la sequenza di transizioni

Stati

- "una condizione o una situazione della vita di un oggetto durante la quale tale oggetto soddisfa una condizione, esegue un'attività o aspetta un evento" [UML]
- Lo stato di un oggetto in qualsiasi momento è determinato da:
 - *I valori dei suoi attributi*
 - *Le relazioni che ha con altri oggetti*
 - *Le attività che sta eseguendo*

Quant Stati?

Colore
rosso: int verde: int blu : int



Sintassi per gli stati

- Le azioni sono istantanee e non interrompibili
- Le transizioni interne occorrono dentro lo stato. No causano la transizione in un nuovo stato
- Le attività richiedono un intervallo di tempo finito e sono interrompibili

Nome dello Stato

Azioni di ingresso e di uscita

Transizioni interne

Attività interna

InserimentoPassword

ingresso/visualizza finestra di dialogo della password

uscita/validazione password

help/visualizza help

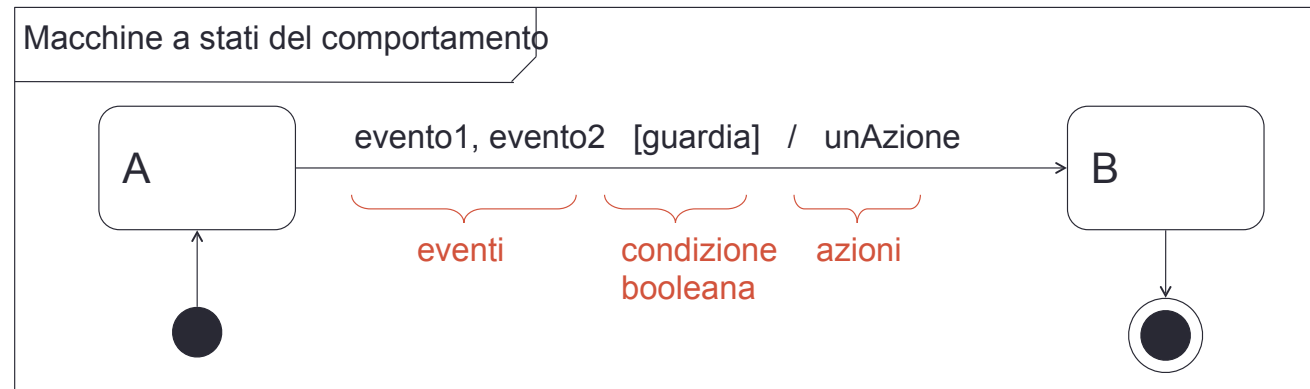
esegui/get password

Sintassi dell'azione: nomeEvento/azione

Sintassi dell'attività: esegui/attività

Transizioni

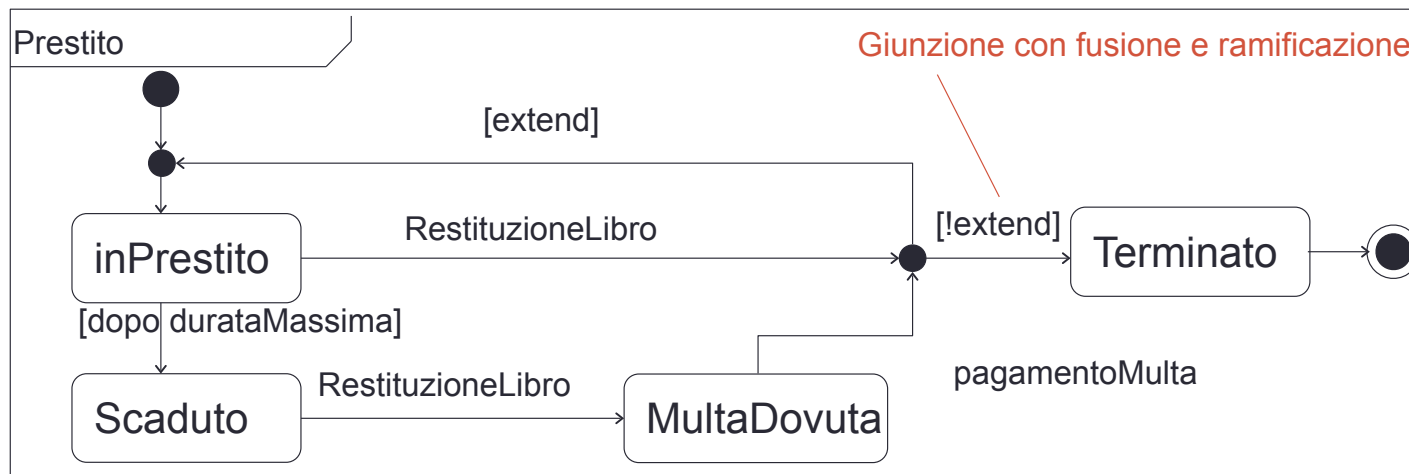
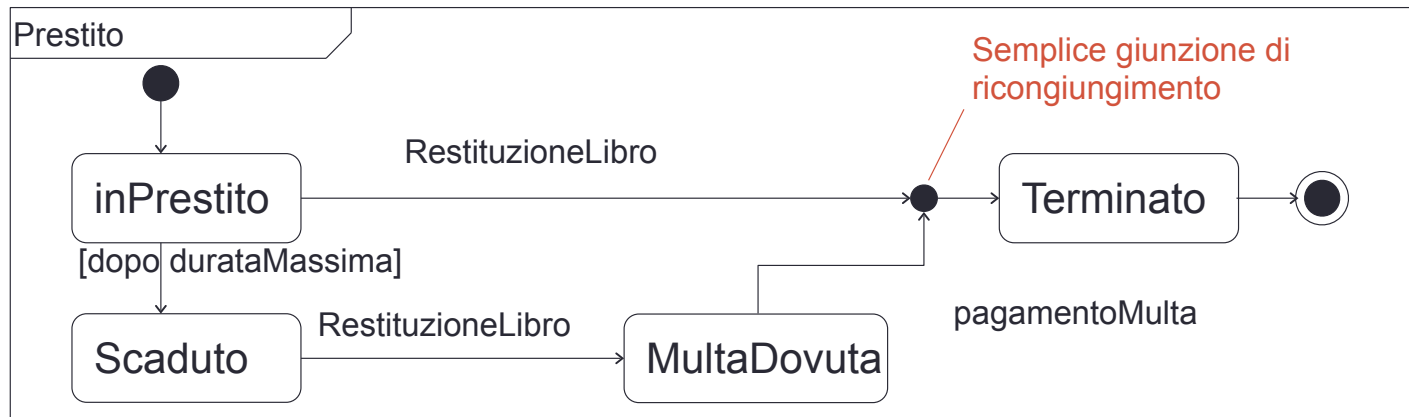
Macchine a stati
del
comportamento



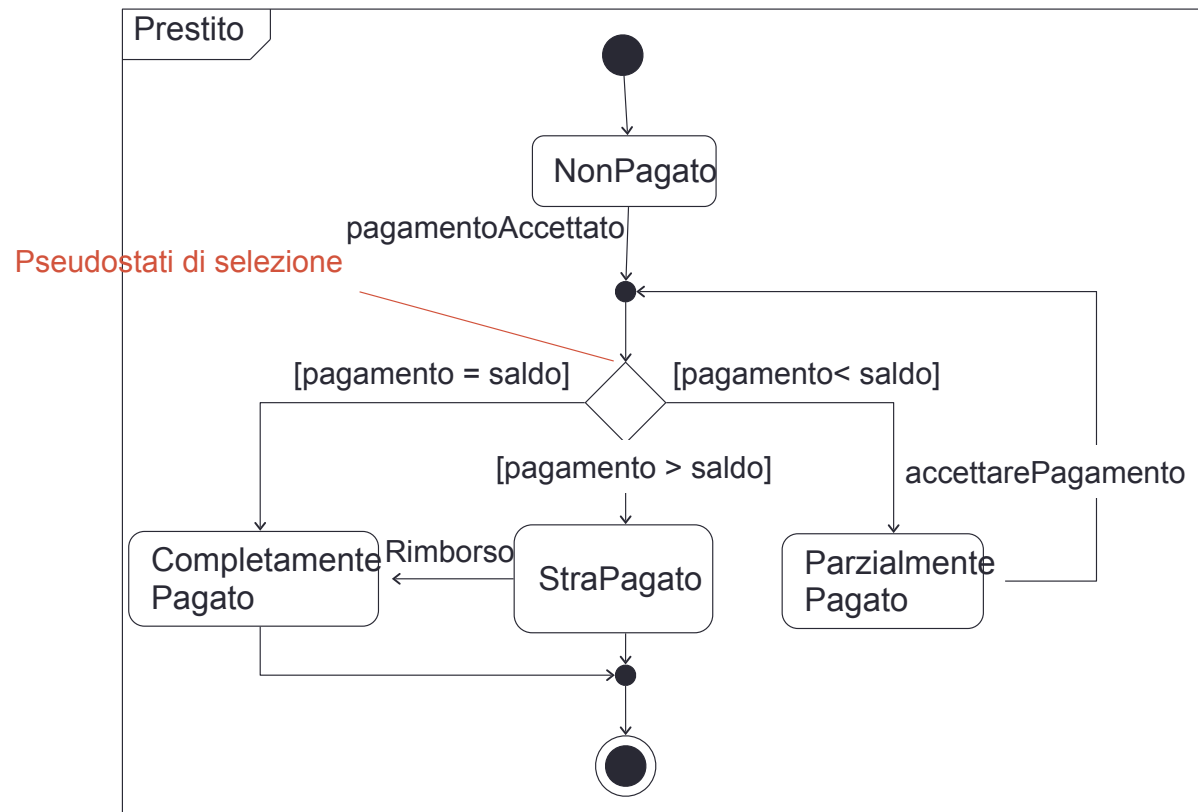
Macchine a stati
del protocollo



Connessione delle transizioni: lo pseudo stato giunzione

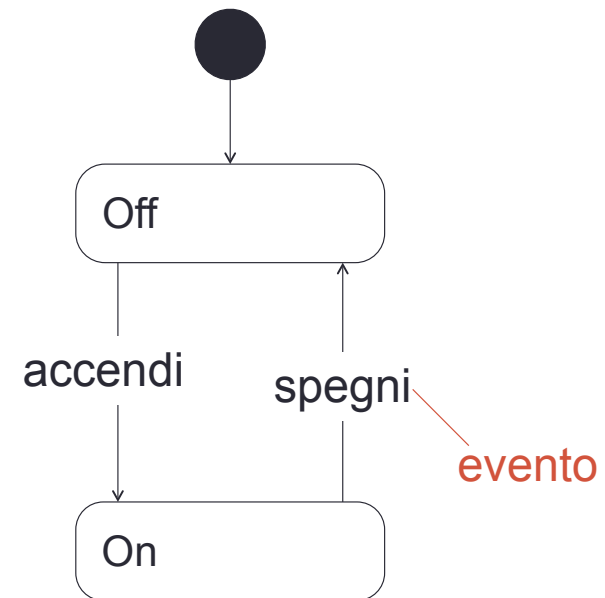


Ramificazione – lo pseudo-stato di selezione



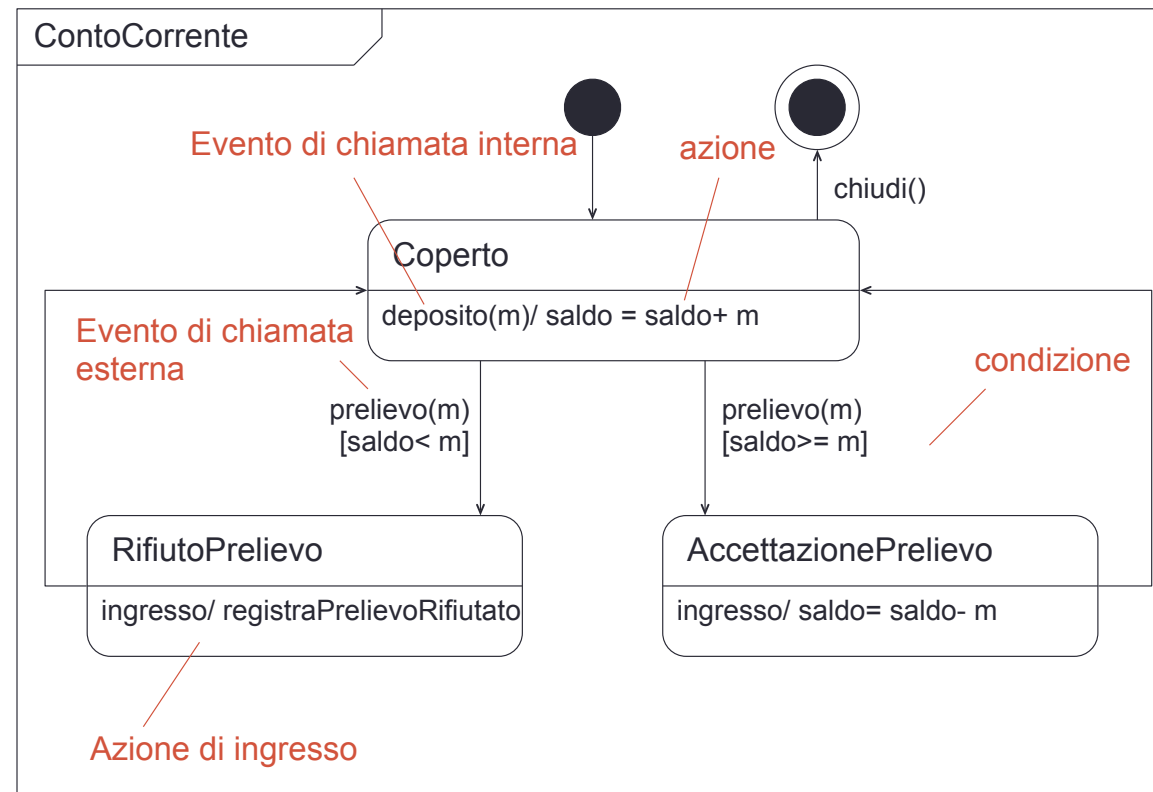
Eventi

- "La specifica di un'occorrenza di interesse che ha una collocazione nel tempo e nello spazio"[UML]
- Gli eventi attivano le transizioni nelle macchine a stati



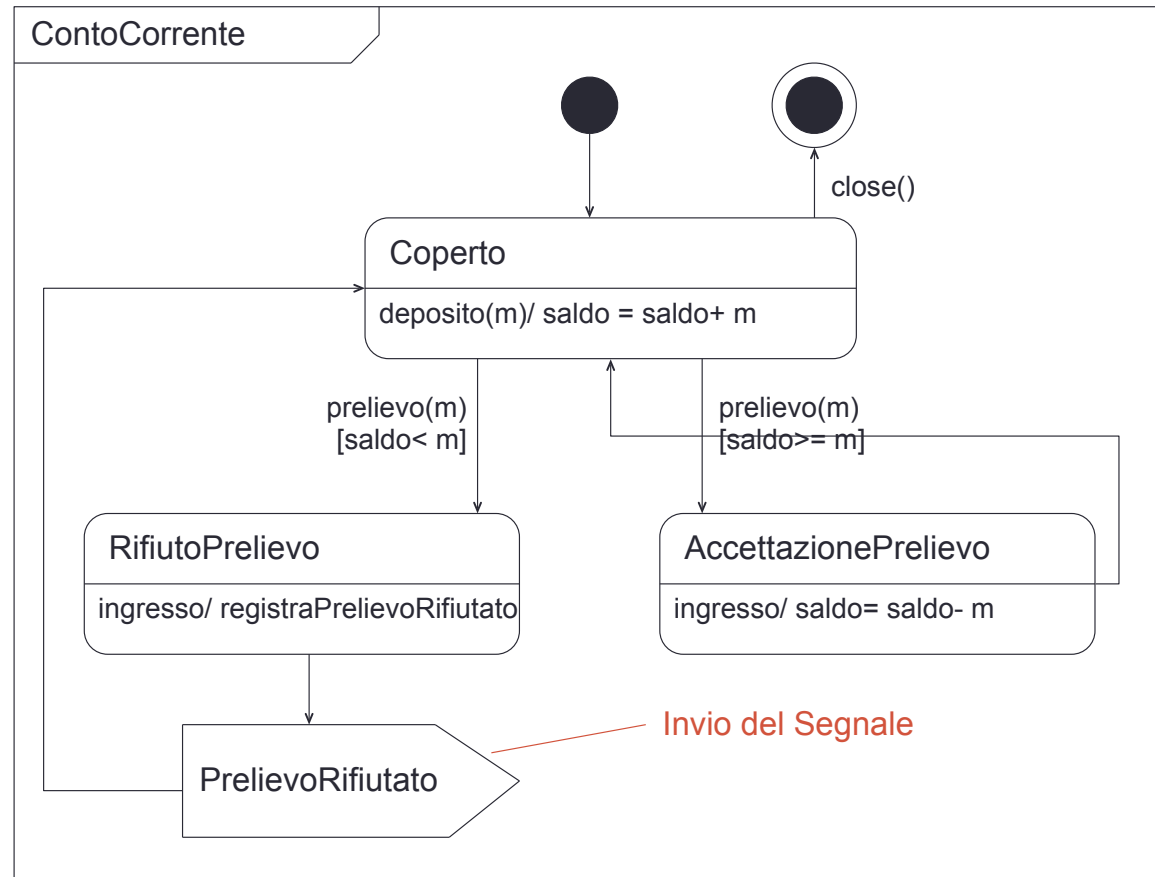
Eventi di Chiamata

- Una chiamata per una specifica operazione
- L'evento dovrebbe avere la stessa segnatura di un'operazione della classe di contesto
- Una sequenza di azioni può essere associata a un evento di chiamata



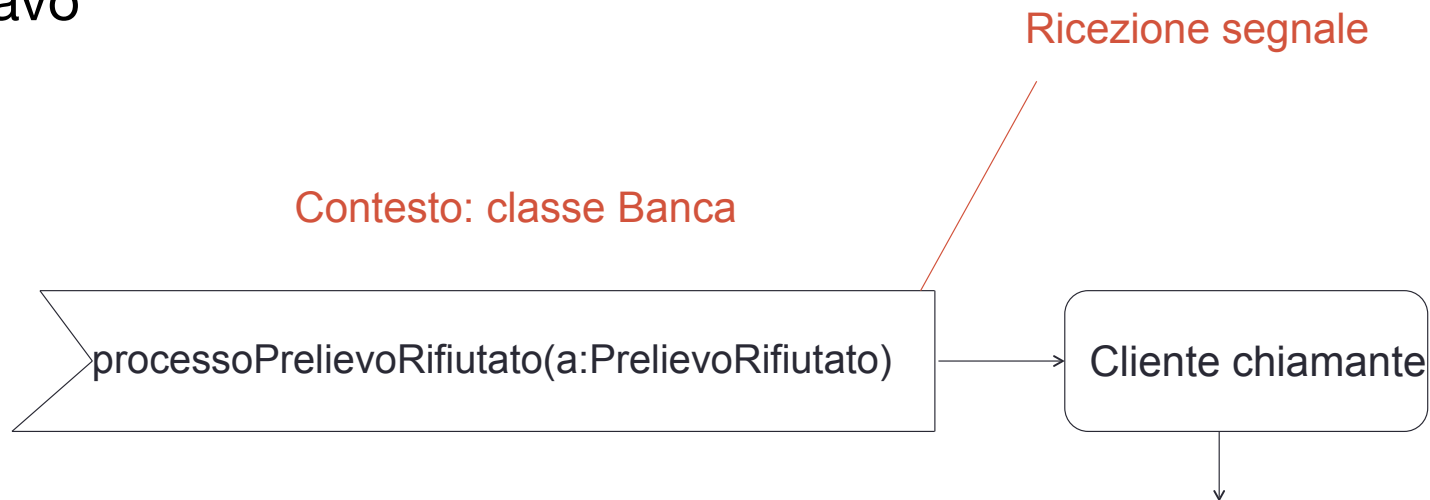
Eventi di segnale

- Un segnale è un pacchetto di informazioni inviato in modo asincrono tra oggetti
 - *Gli attributi del segnale contengono le informazioni*
 - *Non ha operazioni perché il suo scopo è di trasportare informazioni*

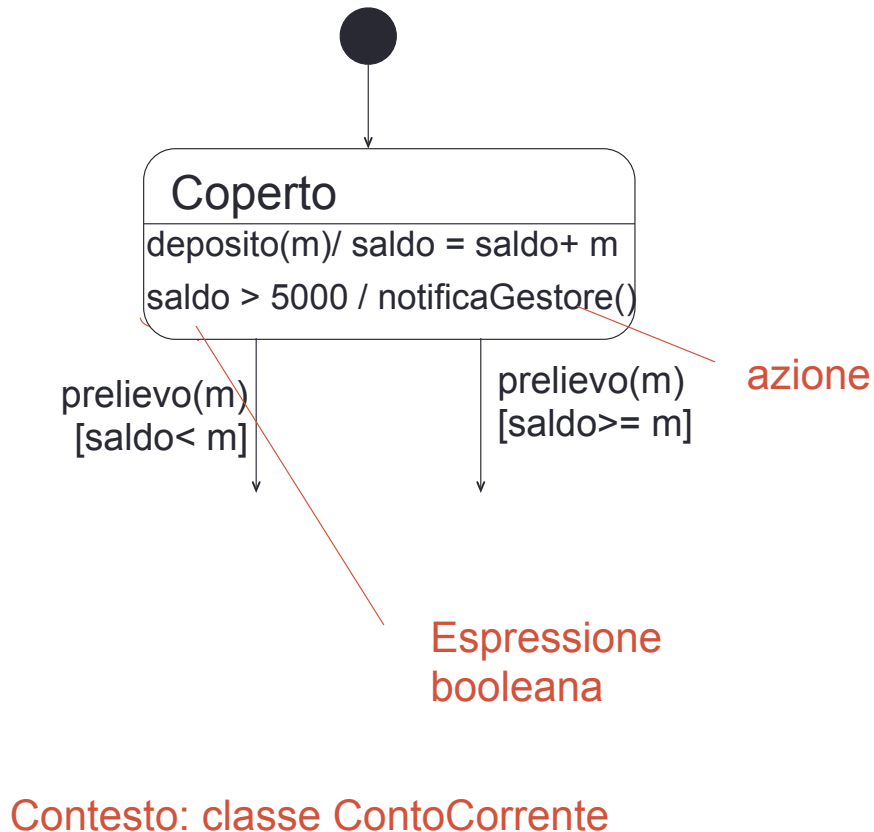


Ricezione del Segnale

- La ricezione di un segnale è indicata da un pentagono concavo



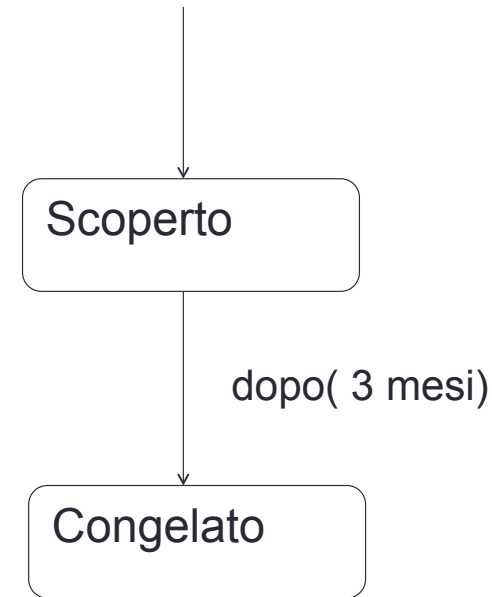
Evento di Variazione



- Un evento di variazione è un'espressione booleana:
 - *L'azione viene eseguita quando il valore dell'espressione passa da falso a vero*
 - *I valori dell'espressione booleana devono essere costanti, variabili globali, attributi o operazioni della classe di contesto*
- Da un punto di vista implementativo, un evento di variazione implica un ciclo di test continuo della condizione booleana per tutta la durata dello stato

Eventi Temporal

- Gli eventi temporali occorrono quando un'espressione di tempo diventa vera
- Le parole chiavi sono, **dopo** e **quando**
 - *dopo(3 mesi)*
 - *quando(data =20/3/2013)*



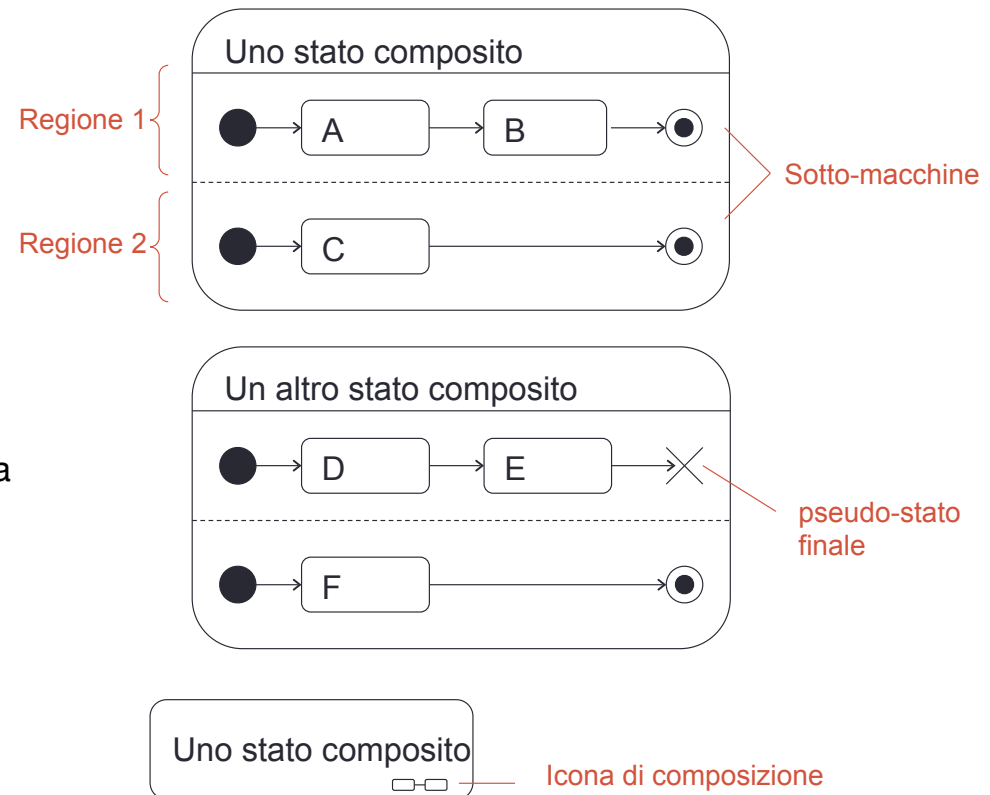
Contesto: classe ContoCredito

Stati Compositi

- Hanno una o più regioni ognuna delle quali contiene una sotto-macchina annidata

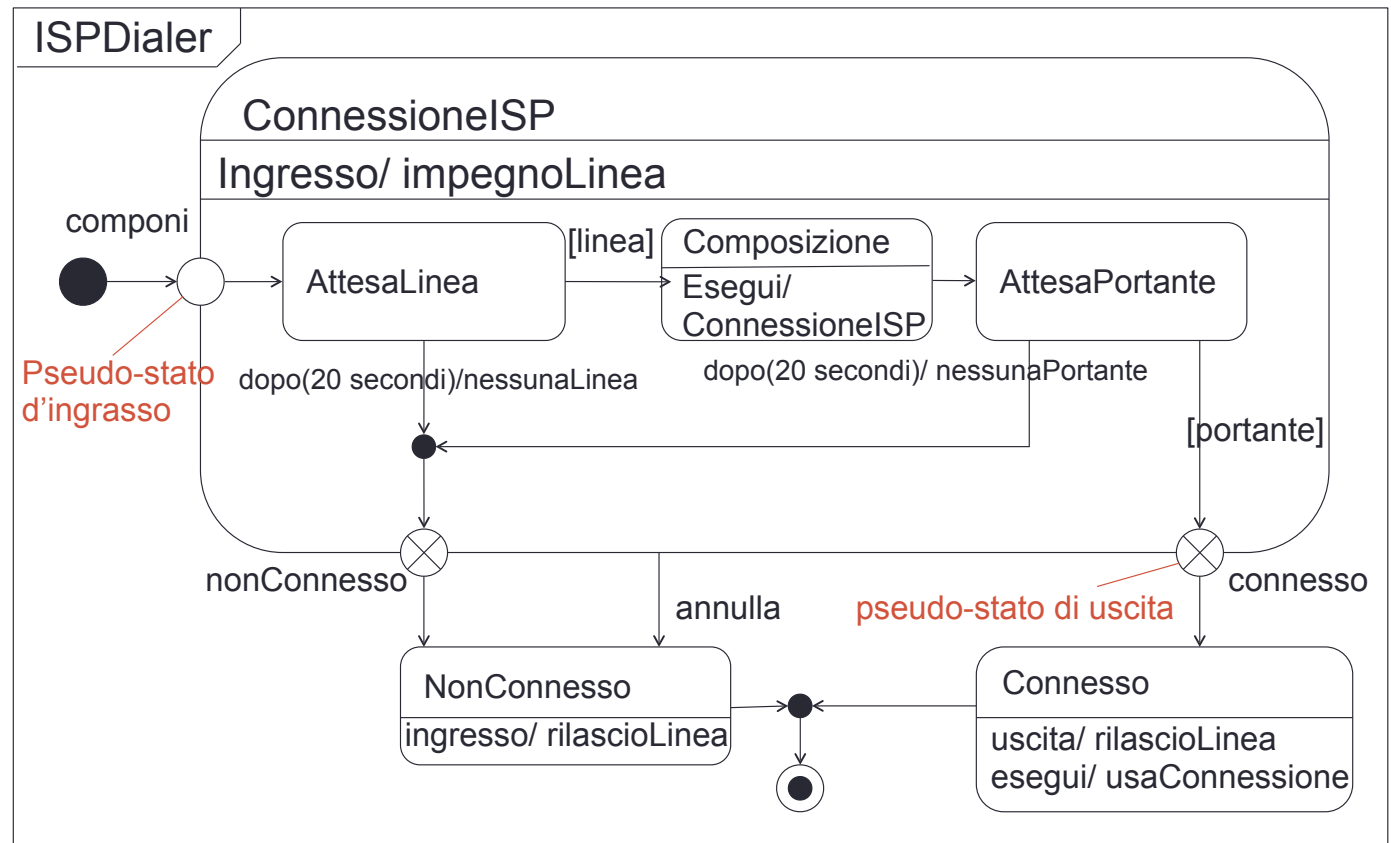
- *Stati composti semplici*
 - *Esattamente una regione*
- *Stati composti ortogonali*
 - *Due o più regioni*

- Lo stato finale viene applicato solo all'interno della regione in cui si trova – le altre regioni continuano la loro esecuzione.
- Lo pseudo-sato finale termina l'esecuzione dell'intero stato composito
- Si usa l'icona di composizione quando le sotto-macchine sono nascoste



Stato Composito semplice

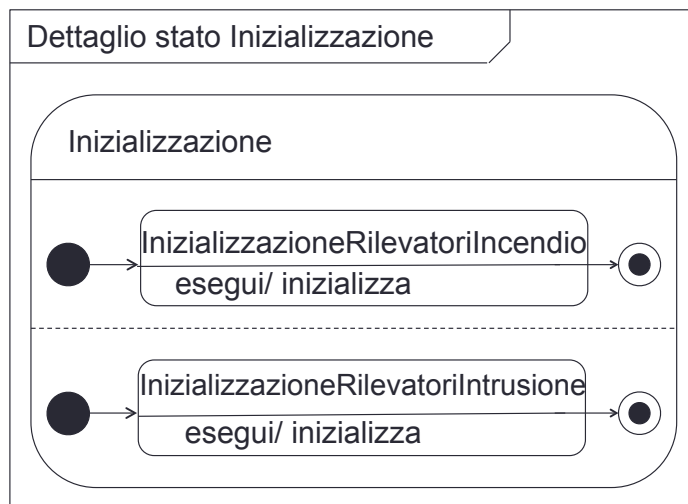
Contiene una singola regione



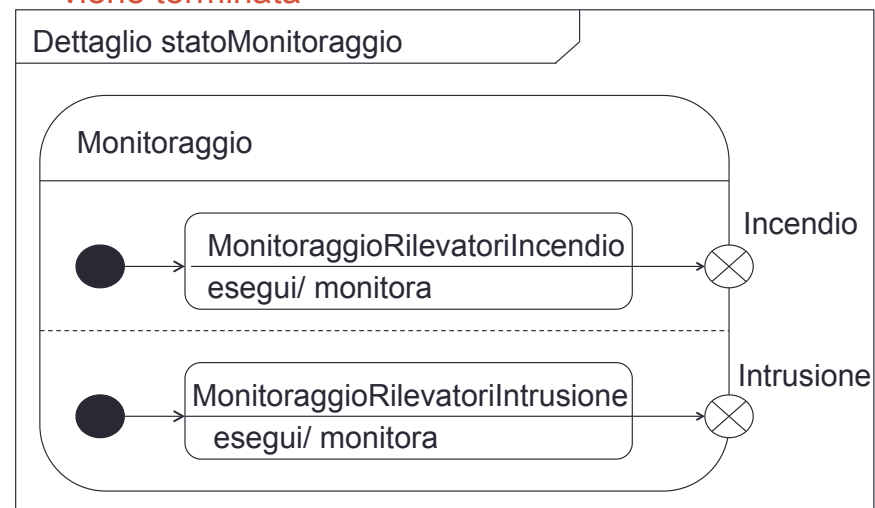
Stati Compositi Ortogonali

- Hanno due o più regioni
- Quando si entra nello stato composito, tutte le sotto-macchine iniziano la loro esecuzione in modo concorrente

Uscita sincronizzata – si esce dallo stato composito quando tutte le regioni sono terminate

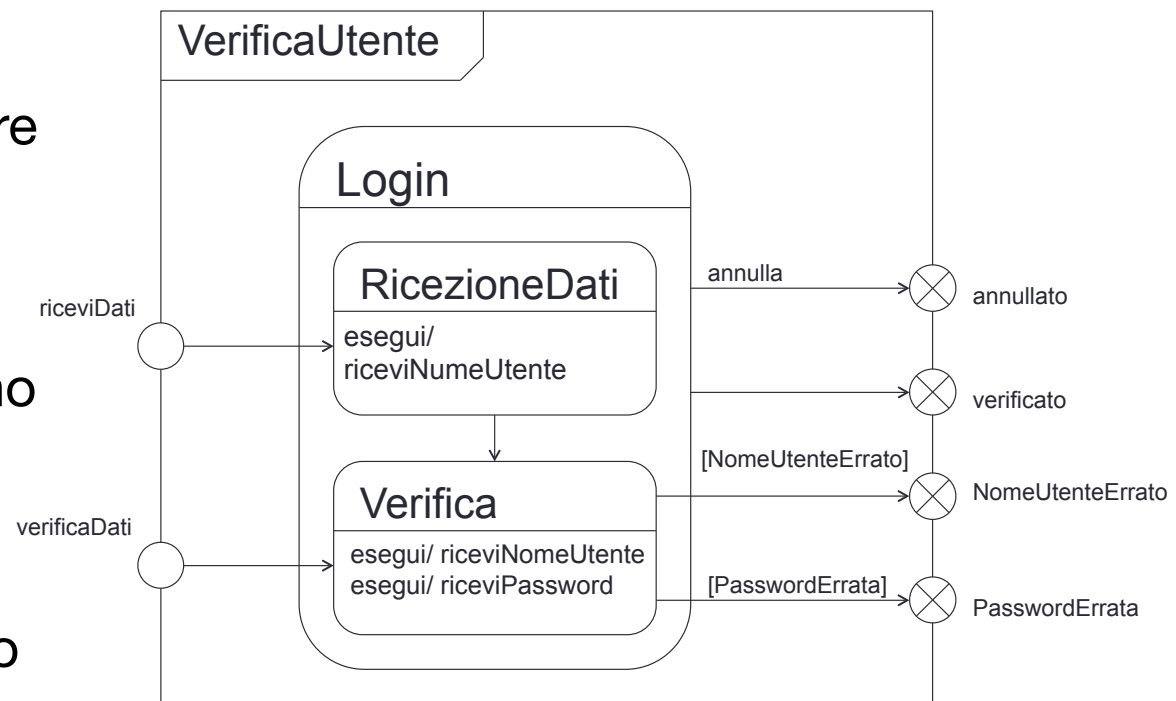


Uscita asincrona - si esce dallo stato composito quando una regione termina. L'altra sotto-macchina viene terminata



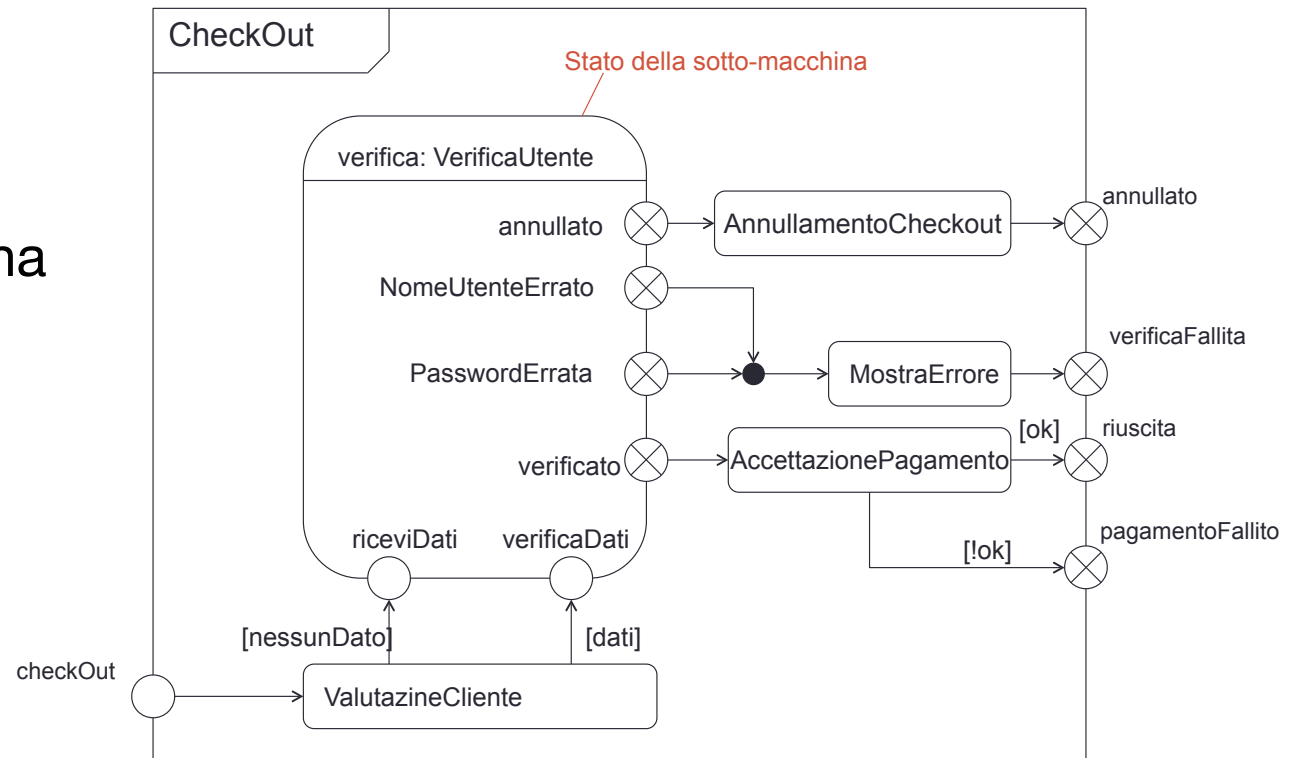
Stati della Sotto-Macchina

- Se vogliamo fare riferimento a questa macchina a stati in altre macchine a stati, senza ingombrare i diagrammi, dobbiamo usare gli *stati della sotto-macchina*
- Gli stati della sotto-macchina fanno riferimento a un'altra macchina a stati
- Gli stati della sotto-macchina sono semanticamente equivalenti agli stati compositi



Sintassi degli Stati della Sotto-Macchina

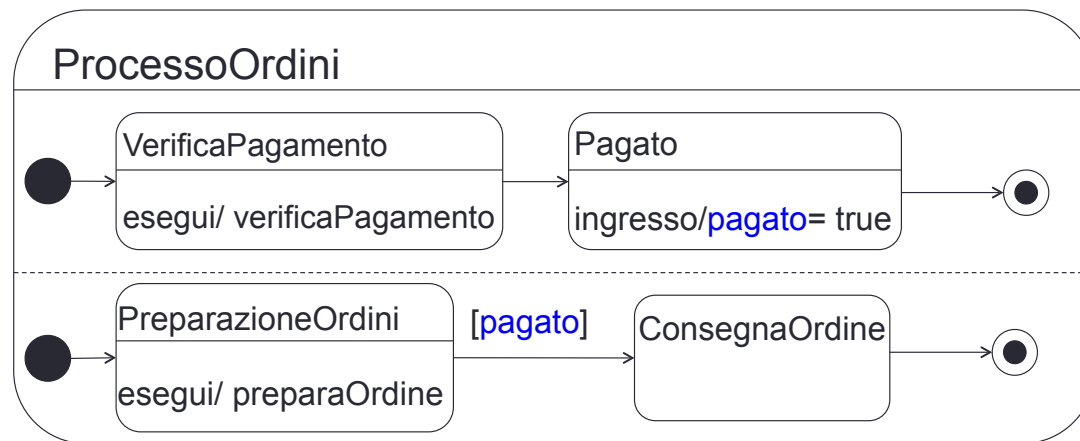
Uno stato della sotto-macchina
è equivalente a includere una
copia della sotto-macchina al
posto dello stato della sotto-
macchina



Comunicazione tra sotto-macchine

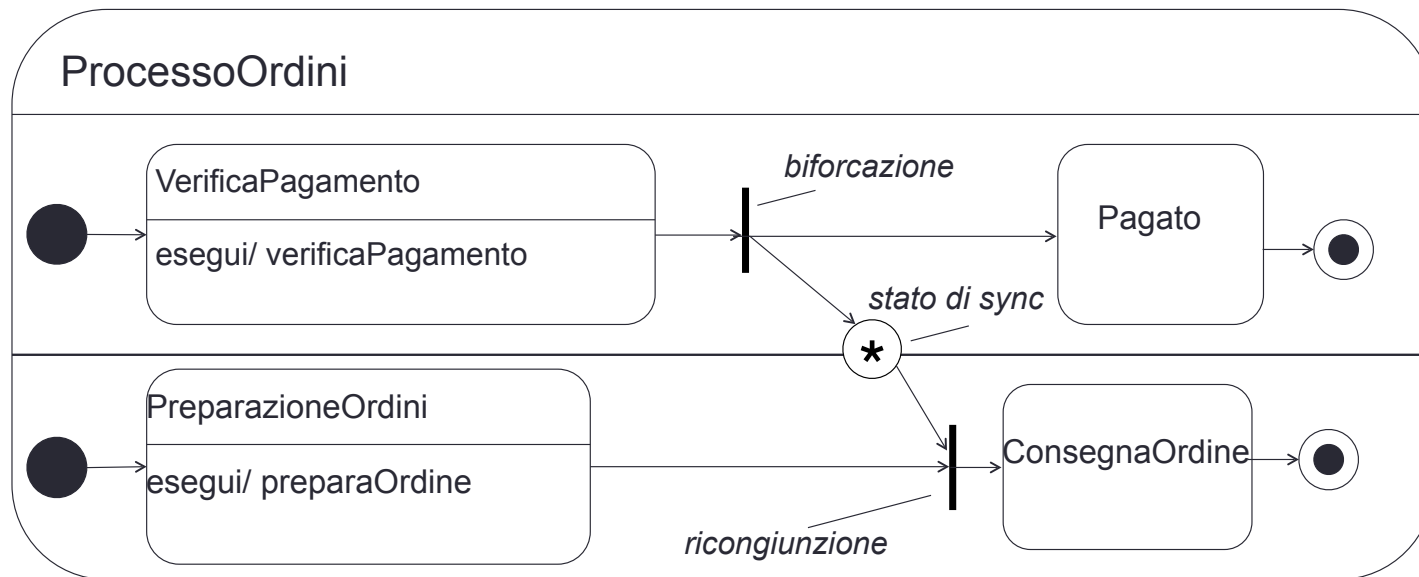
- Capita spesso di avere l'esigenza di far comunicare due sotto-macchine
- Biforcazioni e ricongiunzioni possono essere utilizzate per creare sotto-macchine concorrenti e per ri-sincronizzarle
- La comunicazione asincrona è ottenuta da una sotto-macchina configurando un flag per un'altra sotto-macchina
 - *Utilizzare gli attributi come flag nel contesto degli oggetti*

La comunicazione avviene utilizzando l'attributo ***pagato*** come flag: La sotto-macchina in alto setta il flag e la sotto-macchina in basso usa questo come condizione di guardia



Comunicazione tramite stato di sync

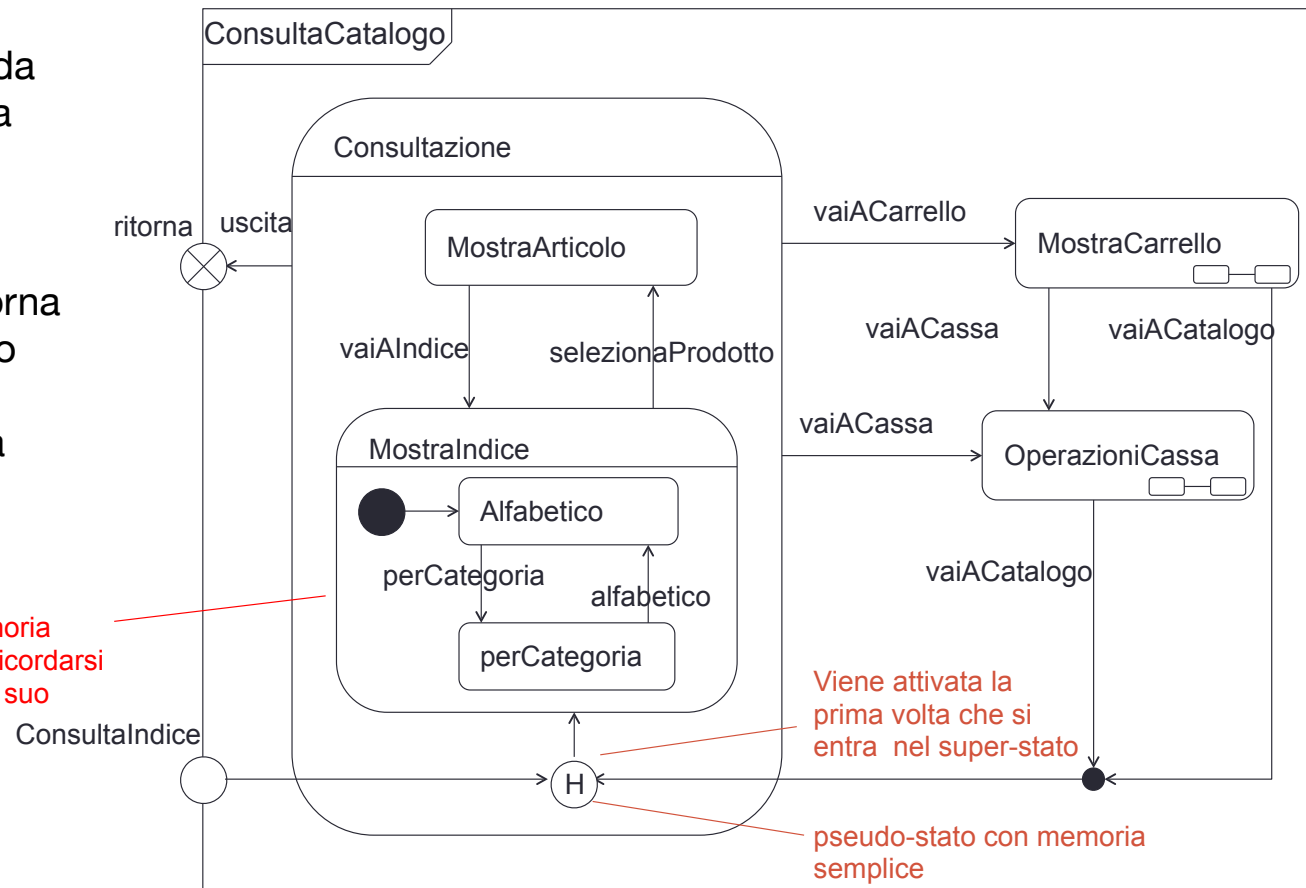
- Lo stato di sync è uno stato speciale il cui compito è quello di tenere traccia di ogni singola attivazione della sua unica transizione di input.
- Lo stato di sync è come una coda: si aggiunge un elemento alla coda ogni volta che viene attivata la transizione di input.
 - *numero limitato di elementi specificando all'interno dello stato di sync un numero intero positivo*
 - *Asterisco per un numero illimitato di elementi*



Stati con memoria semplice

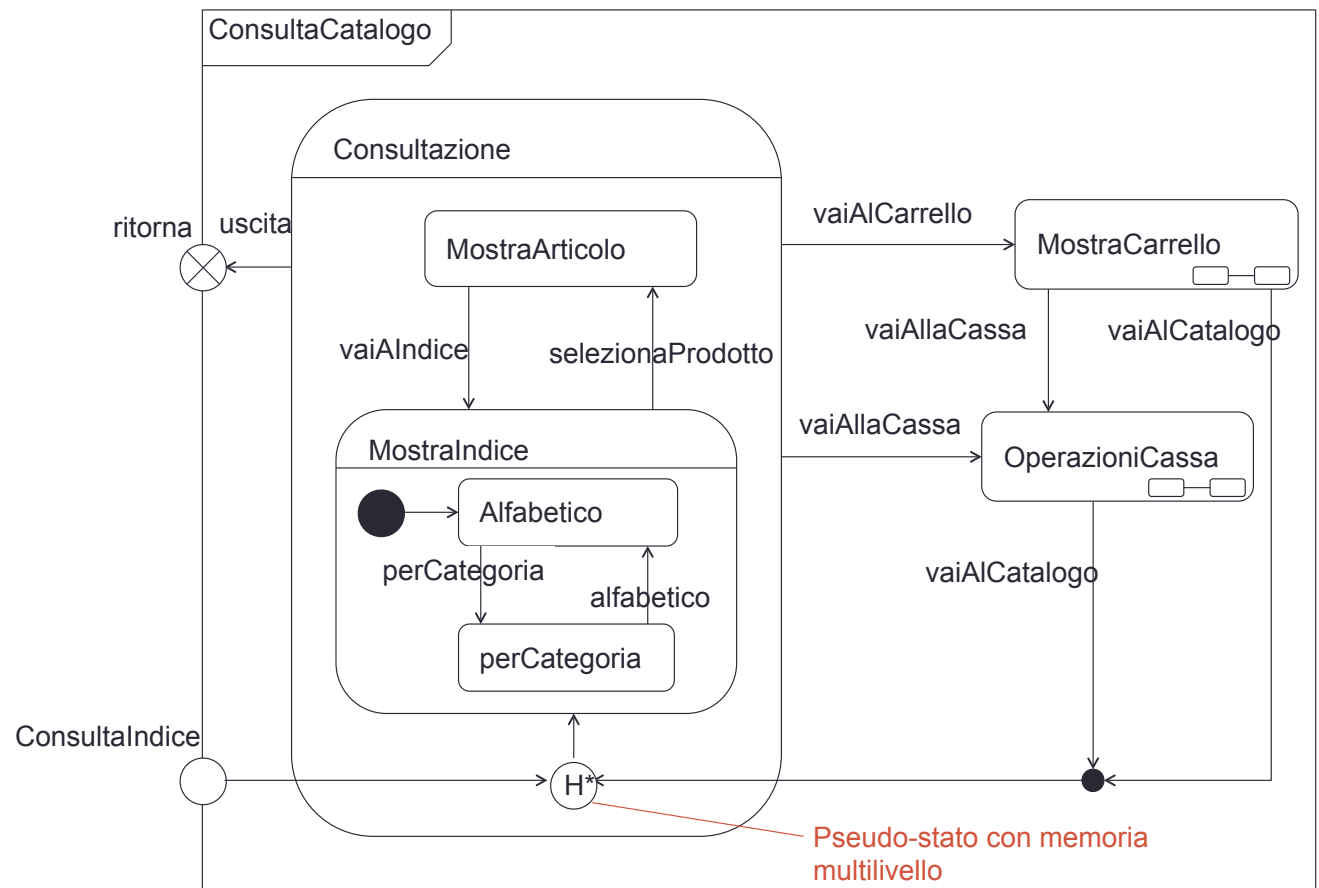
- Lo pseudo-stato con memoria semplice ricorda in quale sottostato si era quando si è lasciato il super-stato
- In seguito quando si ritorna da uno stato esterno allo stato con memoria, l'indicatore ridireziona la transizione sull'ultimo sottostato memorizzato

Uno stato con memoria semplice non può ricordarsi dei sotto-stati di un suo sottostato



Sottostato con memoria multilivello

- Uno stato con memoria multilivello può non solo ricordarsi l'ultimo sottostato attivo allo stesso livello dello pseudo-stato, ma anche l'eventuale ultimo sottostato attivo di questo sottostato attivo... e così via



I diagrammi di macchina a stati in UP

- Non esiste nessun modello in UP chiamato “modello stati”
- Tuttavia qualsiasi elemento in qualsiasi modello può avere una macchina a stati per comprendere o comunicare meglio il proprio comportamento dinamico
 - *Una macchina può essere associata alla classe di progetto Sale del modello di progetto e fa parte anch'essa del modello di progetto*
 - *Un diagramma di macchina stati per mostrare la sequenza valida delle operazioni del caso d'uso Elabora Vendita*

Esempio di macchina a stati per la sequenza valida delle operazioni di un caso di uso

